IT4100

Software Quality Assurance

Group Assignment 1

Member Details

Student Name	Samarathunga S.H.P.N.D
Student ID	IT22576484
Student Email	it22576484@my.sliit.lk
SQA Group ID	SQA-2026-170
Module	Crop JPG module
Automation Tool	Playwright

1. Objective

The objective of this section is to test the **Crop JPG functionality** of the Pixelssuite web application using **Playwright automation along with manual verification**.
The testing focuses on validating key user interactions, including page accessibility, file upload functionality, crop area behavior, preview updates, download functionality, and reset behavior using the Clear option.
Additionally, the goal is to ensure that the user interface responds correctly and that all core features operate reliably under normal usage conditions.

2. Test Environment

- **Browser:** Google Chrome (Chromium-based via Playwright)
- **Target URL:** https://www.pixelssuite.com/crop-jpg
- **Testing Type:** Functional Testing, Smoke Testing, and Basic Regression Testing
- **Automation Tool:** Playwright (JavaScript)
- **Development Environment:** Visual Studio Code

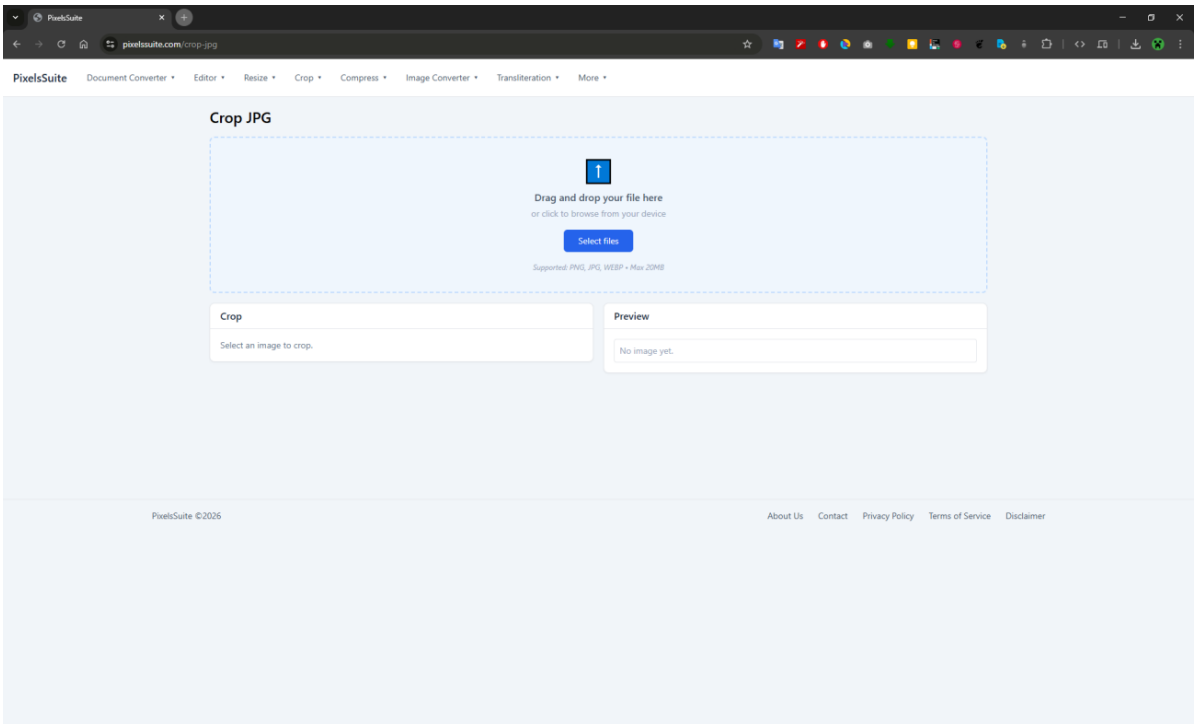
3. Manual Test Cases for the Crop Module

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
M/T-01	Access Crop JPG page	• Open site and navigate to Crop > JPG page.	Crop JPG page loads successfully.	Page loaded successfully and main controls were visible.	Pass

M/T-02	Verify initial UI state	• Observe upload panel, Crop area, and Preview area before any file upload.	Upload panel is visible. Crop area asks user to select an image. Preview area is empty.	Initial UI matched the expected empty state.	Pass
M/T-03	Open file selector	• Click **Select files**.	System opens operating system file picker.	File chooser opened successfully.	Pass
M/T-04	Upload valid JPG image	• Select a valid JPG file from the local computer.	Selected image is loaded into the crop panel and preview panel.	Image loaded successfully; crop area and preview were displayed.	Pass
M/T-05	Verify crop value text boxes after upload	• After uploading an image, observe the **X, Y, Width, Height** input fields.	X, Y, Width, and Height text boxes are displayed with current crop values.	The text boxes appeared and showed crop values correctly.	Pass
M/T-06	Edit crop using text boxes	• Change values in X, Y, Width, and Height fields.	Crop area updates according to the entered values and preview	Entered values updated the crop area	Pass

			changes accordingly.	and preview correctly.	
M/T-07	Adjust crop region using mouse	• Drag the crop selection and resize it using the mouse handles.	Crop selection changes position/size and preview updates accordingly.	After dragging and resizing the crop area, the preview changed correctly.	Pass
M/T-08	Download cropped image	• Click **Download** after setting crop area.	Cropped image is downloaded successfully.	Downloaded file appeared in browser download area as image.jpg.	Pass
M/T-09	Reset using Clear button	• Click **Clear** after upload/crop.	Page returns to the initial empty state and removes loaded image.	Page reset correctly; Crop and Preview panels returned to the default state.	Pass

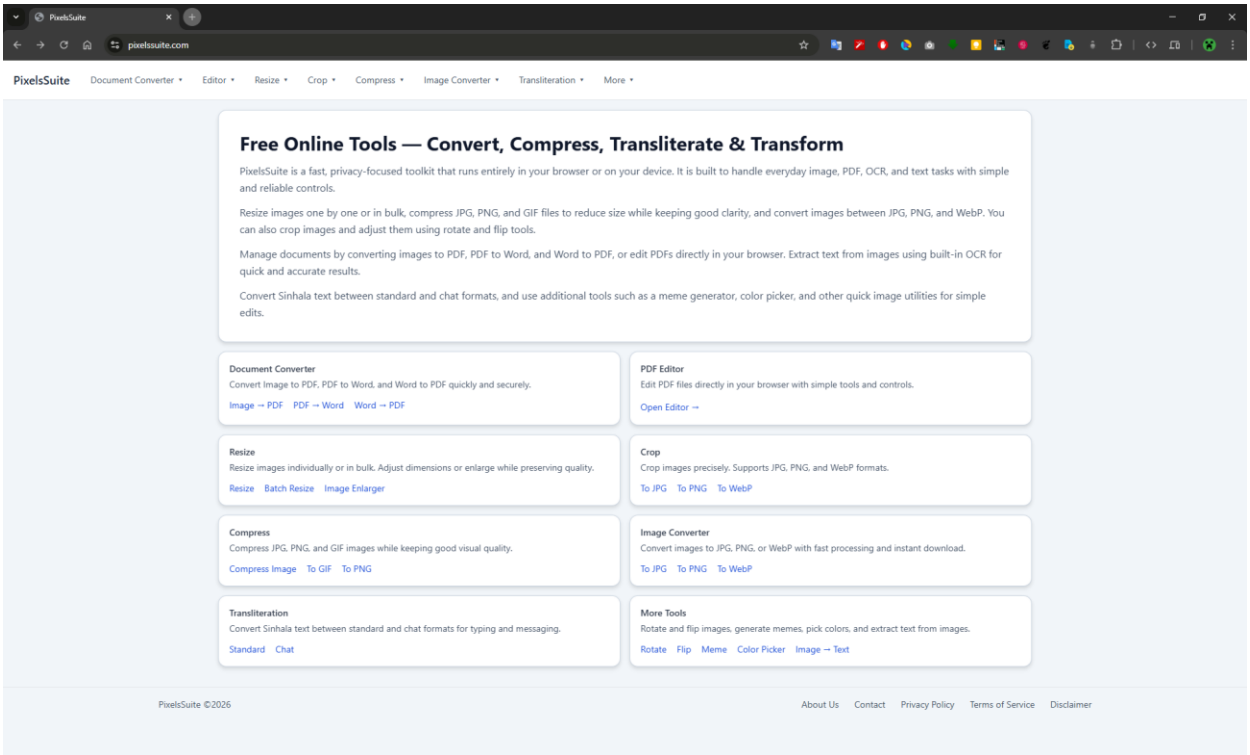
4. Screenshot Evidence



Page returned to initial state after Clear button was pressed.

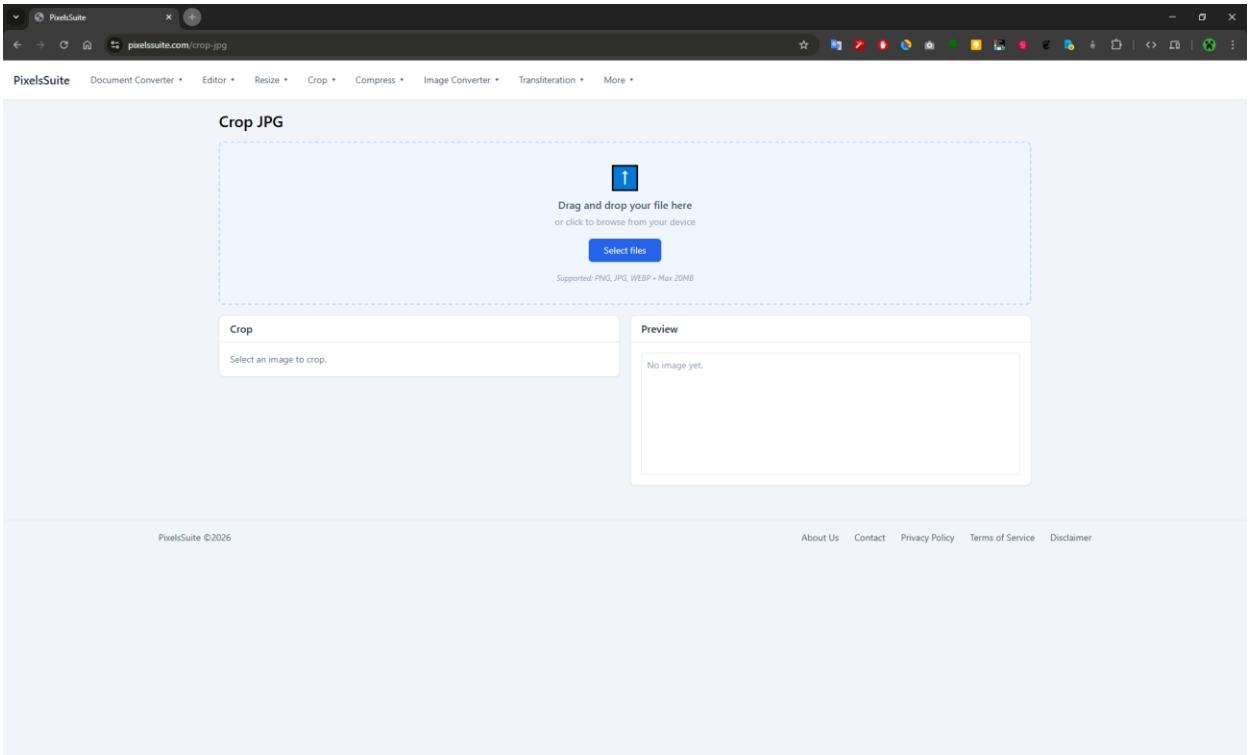
Test Case ID:- M/T-01

Test Scenario :- Access Crop JPG page

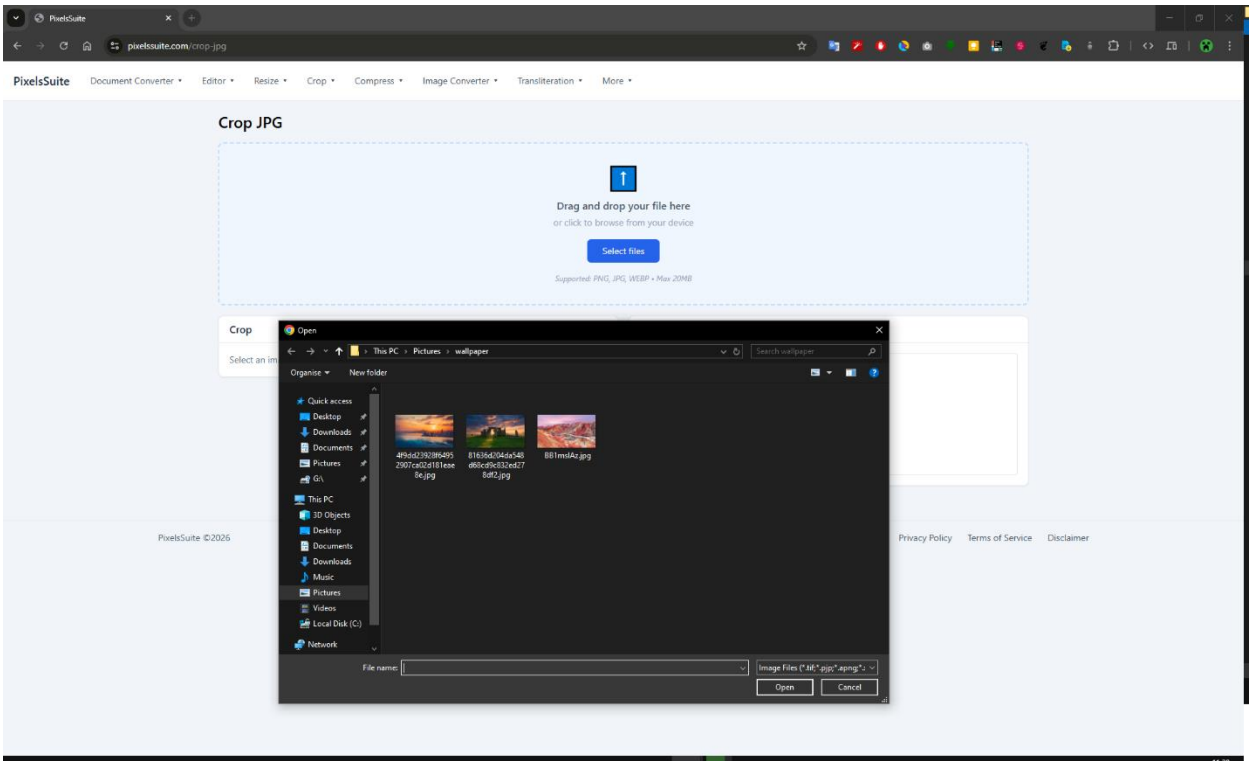


Test Case ID:- M/T-02

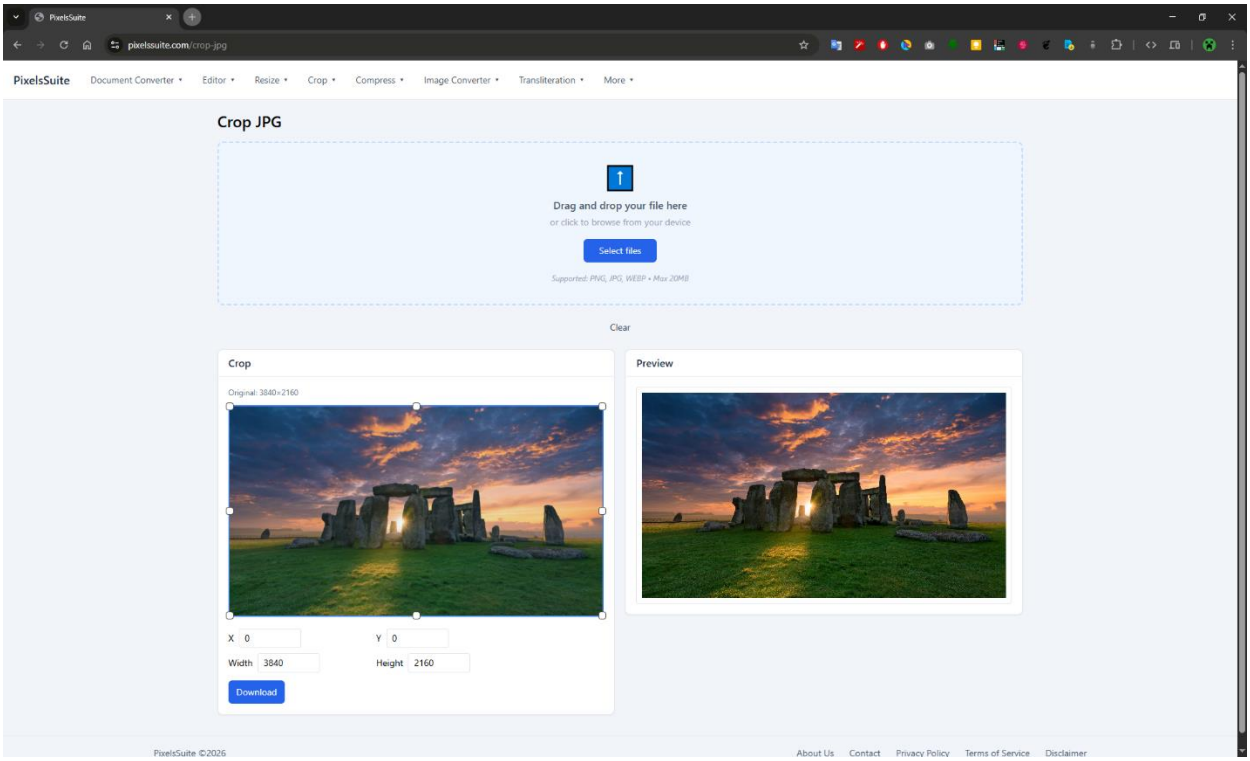
Test Scenario :- Verify initial UI state



Test Case ID:- M/T-03
Test Scenario :- Open file selector

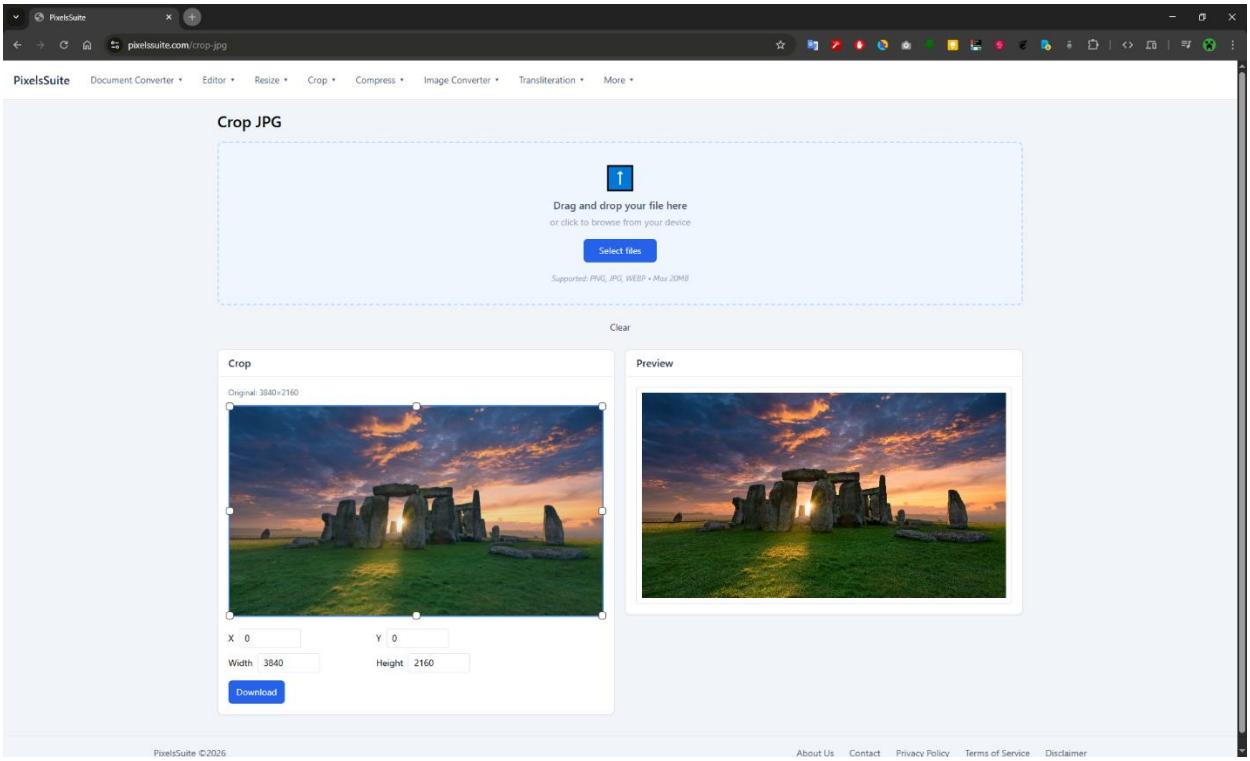


Test Case ID:- M/T-04
Test Scenario :- Upload valid JPG image



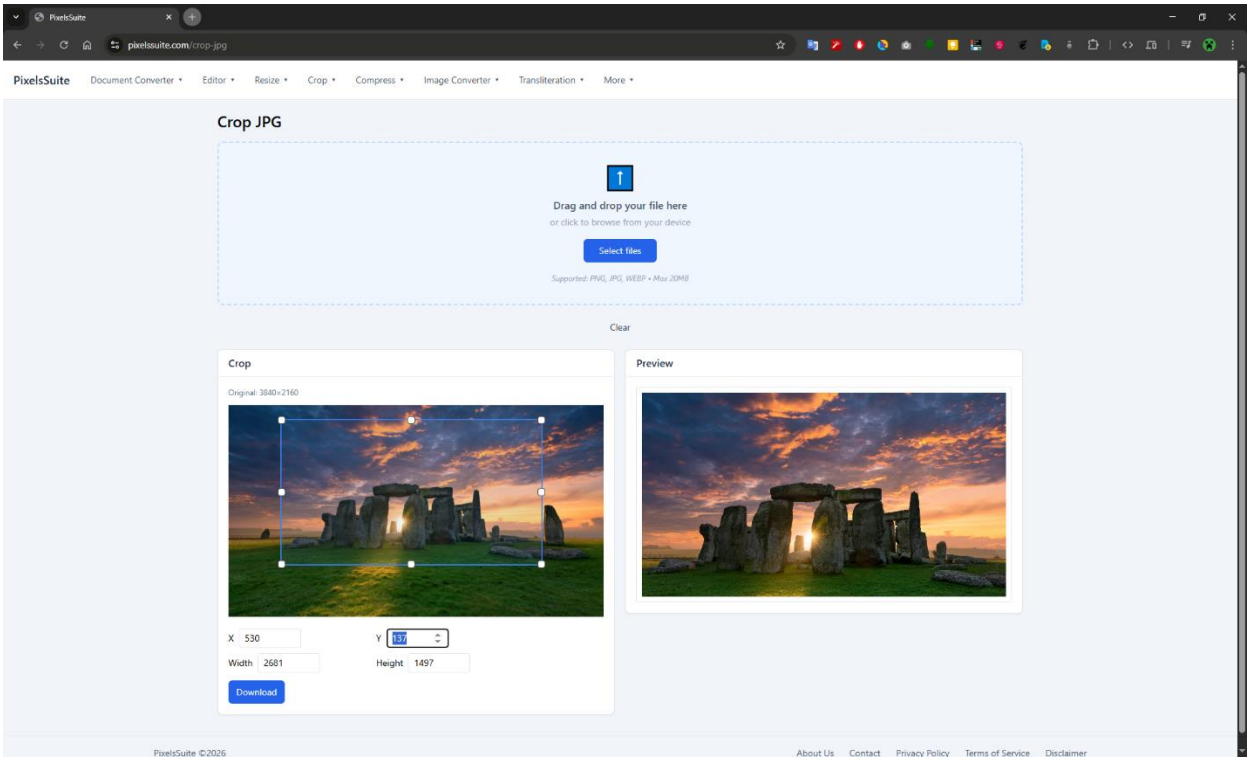
Test Case ID:- M/T-05

Test Scenario :- Verify crop value text boxes after upload



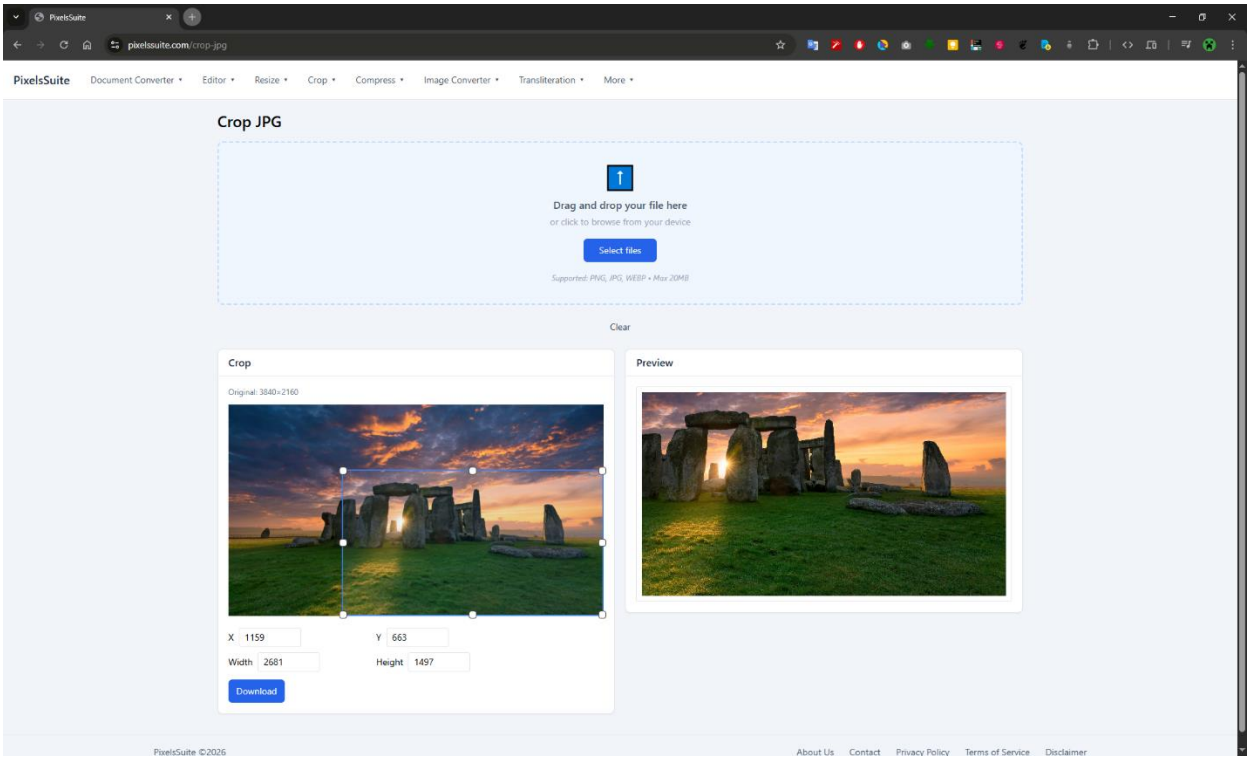
Test Case ID:- M/T-06

Test Scenario :- Edit crop using text boxes

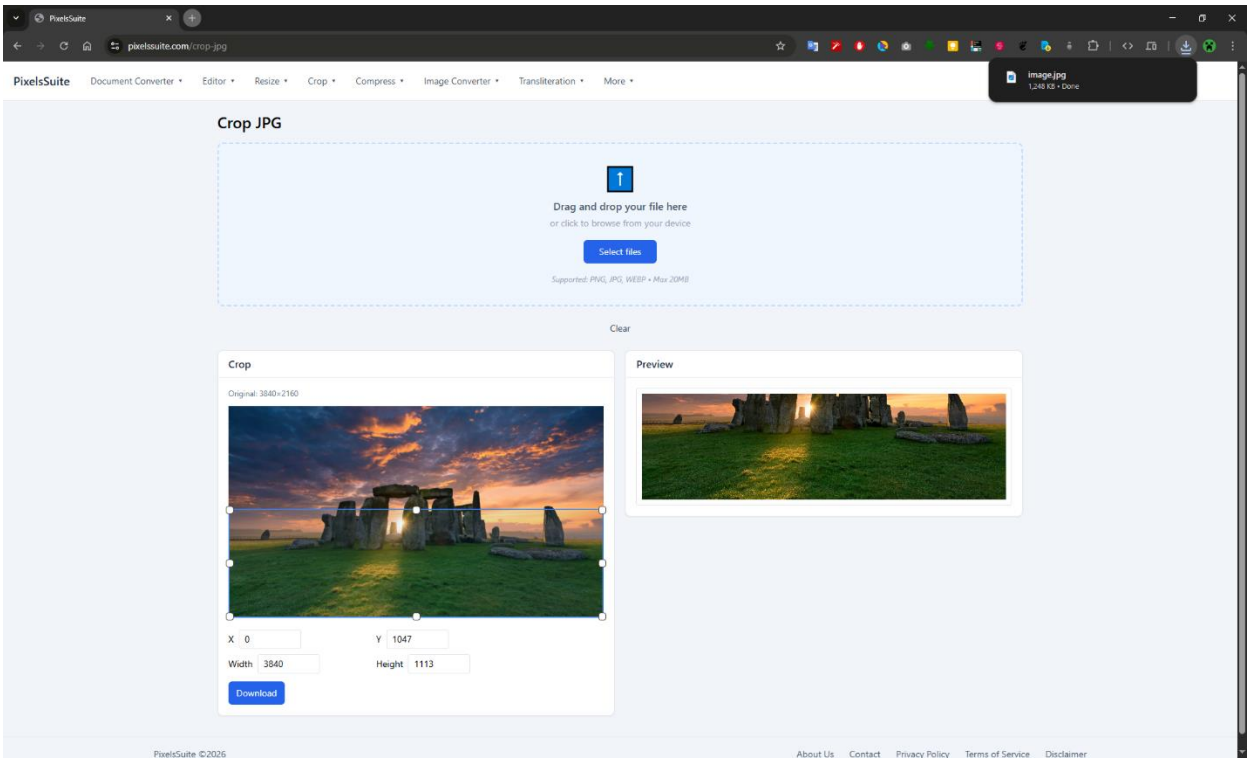


Test Case ID:- M/T-07

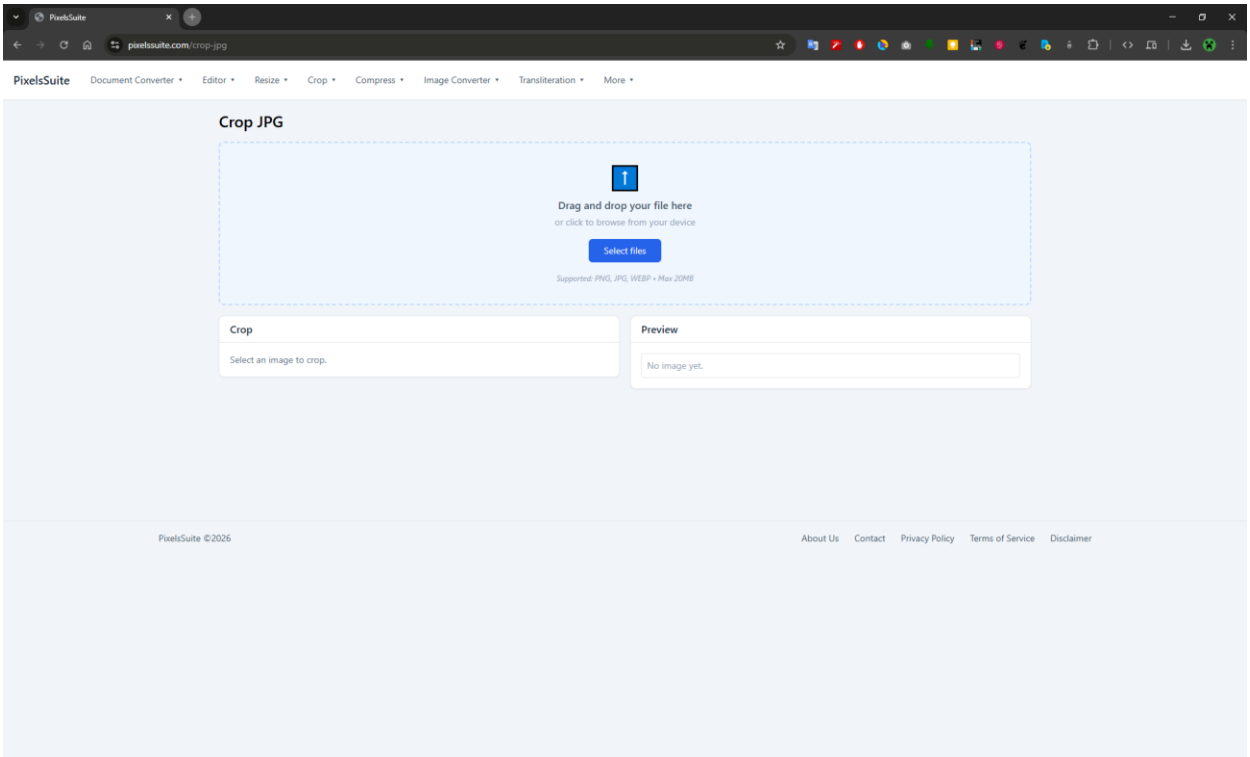
Test Scenario :- Adjust crop region using mouse



Test Case ID:- M/T-08
Test Scenario :- Download cropped image



Test Case ID:- M/T-09
Test Scenario :- Reset using Clear button



5. Automated Testing – Playwright

5.1Code Implementation

The Playwright automation test suite was developed using JavaScript within Visual Studio Code. The test script covers all major functional scenarios of the Crop JPG module, including page loading, file upload, crop value validation, manual editing via text inputs, mouse-based crop adjustments, image download, and reset functionality.

Reusable helper functions and flexible locators were used to improve test stability and maintainability. Assertions were implemented using Playwright’s expect API to validate UI elements and behaviors.

crop_jpg_playwright.spec.js - playwright - Visual Studio Code

PLAYWRIGHT
- node_modules
- playwright-report
- test-results
- tests
  - fixtures
  - crop_jpg_playwright.spe...
- package-lock.json
- package.json
- playwright.config.js

tests > crop_jpg_playwright.spec.js > cropImage

```
91  }
92
93  test('TC_CROP_001 Access Crop JPG page', async ({ page }) => {
94    await openCropPage(page);
95
96    // Screenshot point: successful page entry.
97    await expect(cropHeading(page)).toBeVisible({ timeout: 15000 });
98    await expect(selectFilesButton(page)).toBeVisible();
99    await expect(hiddenFileInput(page).first()).toBeAttached();
100
101   // Confirms the expected route loaded.
102   await expect(page).toHaveURL(/\/crop-jpg$/i);
103 });
104
105 test('TC_CROP_002 Verify initial UI state', async ({ page }) => {
106   await openCropPage(page);
107
108   // Screenshot point: initial/default state before upload.
109   await expect(selectFilesButton(page)).toBeVisible();
110   await expect(originalText(page)).not.toBeVisible();
111
112   // Empty crop/default state checks without relying only on one exact placeholder text.
113   expect(await cropValueInputs(page).count()).toBe(0);
114   await expect(cropImage(page)).not.toBeVisible();
115
116   // Preview should still be in default state before upload.
117   // Some builds keep a canvas node mounted, so we avoid asserting strict DOM absence.
118   await expect(downloadButton(page)).not.toBeVisible();
119   await expect(clearText(page)).not.toBeVisible();
120 });
121
122 test('TC_CROP_003 Open file selector', async ({ page }) => {
123   await openCropPage(page);
124
125   // Intended to open the operating system file picker dialog.
126   // The visible "Select files" label is nested; click its wrapper for reliability.
127   await selectFilesButton(page).first().locator('xpath=..').click({ force: true });
128
129   // Minimal stable assertion after click path.
130   await expect(hiddenFileInput(page).first()).toBeAttached();
131
      Implement with Codex
132   // TODO: native OS file dialogs are outside Playwright DOM assertions.
133   // Upload itself is validated with setInputFiles in later test cases.
134 });
135
136 test('TC_CROP_004 Upload valid JPG image', async ({ page }) => {
137   await openCropPage(page);
138   await uploadFixtureImage(page);
```

Screen Reader Optimized | Ln 46, Col 2 | Spaces: 2 | UTF-8 | LF | JavaScript | Port : 5500 | Prettier

crop_jpg_playwright.spec.js - playwright - Visual Studio Code

PLAYWRIGHT
- node_modules
- playwright-report
- test-results
- tests
  - fixtures
  - crop_jpg_playwright.spe...
- package-lock.json
- package.json
- playwright.config.js

tests > crop_jpg_playwright.spec.js > cropImage

```
122 test('TC_CROP_003 Open file selector', async ({ page }) => {
133   // Upload itself is validated with setInputFiles in later test cases.
134 });
135
136 test('TC_CROP_004 Upload valid JPG image', async ({ page }) => {
137   await openCropPage(page);
138   await uploadFixtureImage(page);
139
140   // Screenshot point: upload result in crop + preview panes.
141   await expect(originalText(page)).toBeVisible();
142   await expect(cropImage(page)).toBeVisible();
143   await expect(cropSelectionBox(page)).toBeVisible();
144   await expect(previewCanvas(page)).toBeVisible();
145   await expect(clearText(page)).toBeVisible();
146 });
147
148 test('TC_CROP_005 Verify crop value text boxes after upload', async ({ page }) => {
149   await openCropPage(page);
150   await uploadFixtureImage(page);
151
152   const { xInput, yInput, widthInput, heightInput } = await getFirstFourVisibleCropInputs(page);
153
154   // Screenshot point: the first four non-file inputs interpreted as X/Y/Width/Height.
155   await expect(xInput).toBeVisible();
156   await expect(yInput).toBeVisible();
157   await expect(widthInput).toBeVisible();
158   await expect(heightInput).toBeVisible();
159
160   await expect(xInput).toHaveValue(/.+/);
161   await expect(yInput).toHaveValue(/.+/);
162   await expect(widthInput).toHaveValue(/.+/);
163   await expect(heightInput).toHaveValue(/.+/);
164 });
165
166 test('TC_CROP_006 Edit crop using text boxes', async ({ page }) => {
167   await openCropPage(page);
168   await uploadFixtureImage(page);
169
170   const { xInput, yInput, widthInput, heightInput } = await getFirstFourVisibleCropInputs(page);
171
172   // Screenshot point: manual crop values edited via text boxes.
173   await xInput.fill('5');
174   await expect(xInput).toHaveValue('5');
175
176   await yInput.fill('5');
177   await expect(yInput).toHaveValue('5');
178
179   await widthInput.fill('70');
180   await expect(widthInput).toHaveValue('70');
```

Screen Reader Optimized | Ln 46, Col 2 | Spaces: 2 | UTF-8 | LF | JavaScript | Port : 5500 | Prettier

crop_jpg_playwright.spec.js - playwright - Visual Studio Code

PLAYWRIGHT
- node_modules
- playwright-report
- test-results
- tests
  - fixtures
  - crop_jpg_playwright.spe...
- package-lock.json
- package.json
- playwright.config.js

tests > crop_jpg_playwright.spec.js > cropImage

```
148 test('TC_CROP_005 Verify crop value text boxes after upload', async ({ page }) => {
162   await expect(widthInput).toHaveValue(/.+/);
163   await expect(heightInput).toHaveValue(/.+/);
164 });
165
166 test('TC_CROP_006 Edit crop using text boxes', async ({ page }) => {
167   await openCropPage(page);
168   await uploadFixtureImage(page);
169
170   const { xInput, yInput, widthInput, heightInput } = await getFirstFourVisibleCropInputs(page);
171
172   // Screenshot point: manual crop values edited via text boxes.
173   await xInput.fill('5');
174   await expect(xInput).toHaveValue('5');
175
176   await yInput.fill('5');
177   await expect(yInput).toHaveValue('5');
178
179   await widthInput.fill('70');
180   await expect(widthInput).toHaveValue('70');
181
182   await heightInput.fill('50');
183   await expect(heightInput).toHaveValue('50');
184 });
185
186 test('TC_CROP_007 Adjust crop region using mouse', async ({ page }) => {
187   await openCropPage(page);
188   await uploadFixtureImage(page);
189
190   // Reduce selection first so mouse drag can visibly move crop coordinates.
191   const { widthInput, heightInput } = await getFirstFourVisibleCropInputs(page);
192   await widthInput.fill('70');
193   await heightInput.fill('50');
194
195   await expect(cropSelectionBox(page)).toBeVisible();
196   const before = await readCropValues(page);
197
      Implement with Codex
198   // TODO: if this move-box locator changes, replace with a dedicated crop selection test id.
199   const selectionBox = cropSelectionBox(page);
200   const selectionBounds = await selectionBox.boundingBox();
201   expect(selectionBounds).toBeTruthy();
202
203   // Screenshot point: moving crop selection with mouse drag.
204   await page.mouse.move(
205     selectionBounds.x + selectionBounds.width / 2,
206     selectionBounds.y + selectionBounds.height / 2
207   );
208   await page.mouse.down();
```

Screen Reader Optimized | Ln 46, Col 2 | Spaces: 2 | UTF-8 | LF | JavaScript | Port : 5500 | Prettier

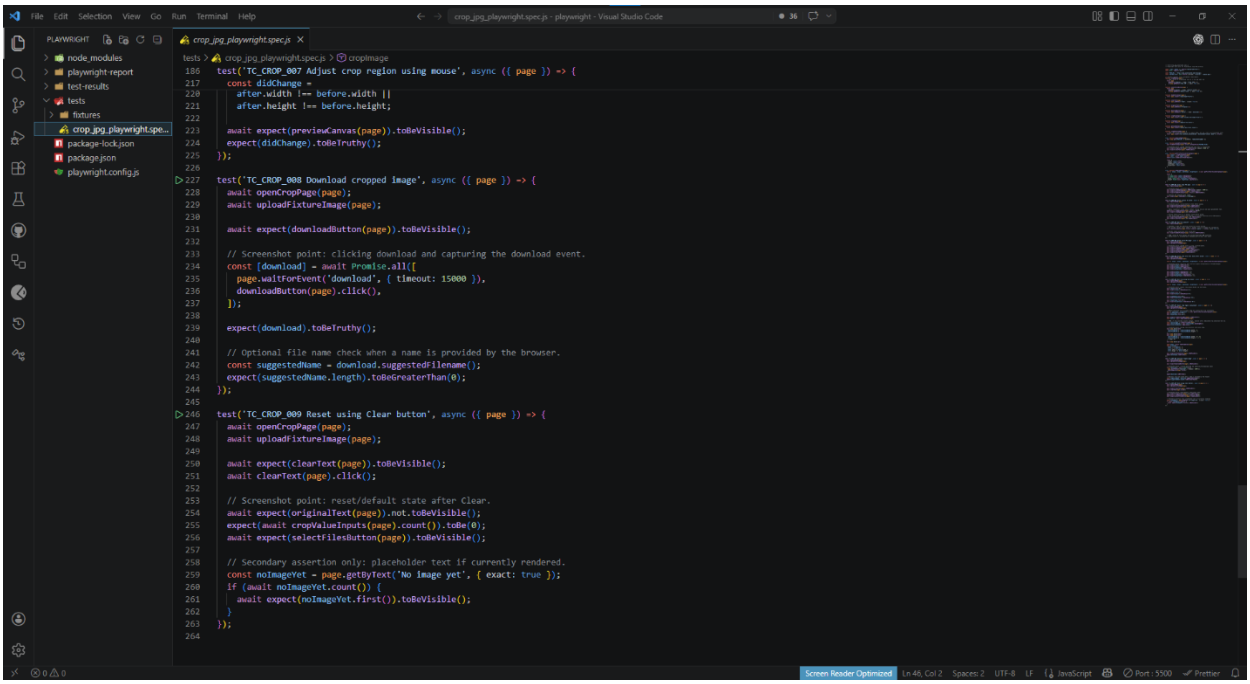
crop_jpg_playwright.spec.js - playwright - Visual Studio Code

PLAYWRIGHT
- node_modules
- playwright-report
- test-results
- tests
  - fixtures
  - crop_jpg_playwright.spe...
- package-lock.json
- package.json
- playwright.config.js

tests > crop_jpg_playwright.spec.js > cropImage

```
166 test('TC_CROP_006 Edit crop using text boxes', async ({ page }) => {
182   await heightInput.fill('50');
183   await expect(heightInput).toHaveValue('50');
184 });
185
186 test('TC_CROP_007 Adjust crop region using mouse', async ({ page }) => {
187   await openCropPage(page);
188   await uploadFixtureImage(page);
189
190   // Reduce selection first so mouse drag can visibly move crop coordinates.
191   const { widthInput, heightInput } = await getFirstFourVisibleCropInputs(page);
192   await widthInput.fill('70');
193   await heightInput.fill('50');
194
195   await expect(cropSelectionBox(page)).toBeVisible();
196   const before = await readCropValues(page);
197
      Implement with Codex
198   // TODO: if this move-box locator changes, replace with a dedicated crop selection test id.
199   const selectionBox = cropSelectionBox(page);
200   const selectionBounds = await selectionBox.boundingBox();
201   expect(selectionBounds).toBeTruthy();
202
203   // Screenshot point: moving crop selection with mouse drag.
204   await page.mouse.move(
205     selectionBounds.x + selectionBounds.width / 2,
206     selectionBounds.y + selectionBounds.height / 2
207   );
208   await page.mouse.down();
209   await page.mouse.move(
210     selectionBounds.x + selectionBounds.width / 2 + 10,
211     selectionBounds.y + selectionBounds.height / 2 + 8,
212     { steps: 8 }
213   );
214   await page.mouse.up();
215
216   const after = await readCropValues(page);
217   const didChange =
218     after.x !== before.x ||
219     after.y !== before.y ||
220     after.width !== before.width ||
221     after.height !== before.height;
222
223   await expect(previewCanvas(page)).toBeVisible();
224   expect(didChange).toBeTruthy();
225 });
226
```

Screen Reader Optimized | Ln 48, Col 2 | Spaces: 2 | UTF-8 | LF | JavaScript | Port : 5500 | Prettier

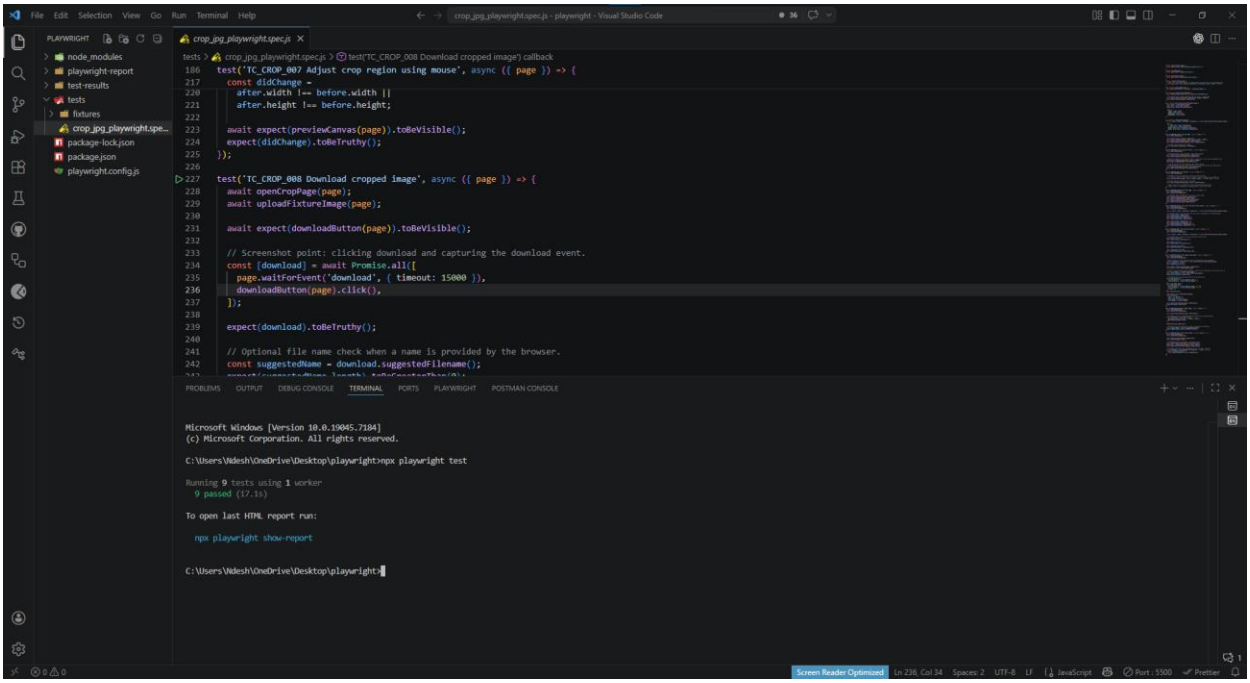


```
test('TC_CROP_007 Adjust crop region using mouse', async ({ page }) => {
  const didChange =
    after.width !== before.width ||
    after.height !== before.height;

  await expect(previewCanvas(page)).toBeVisible();
  expect(didChange).toBeTruthy();
});

test('TC_CROP_008 Download cropped image', async ({ page }) => {
  await openCropPage(page);
  await uploadFixtureImage(page);

  await expect(downloadButton(page)).toBeVisible();

  // Screenshot point: clicking download and capturing the download event.
  const [download] = await Promise.all([
    page.waitForEvent('download', { timeout: 15000 }),
    downloadButton(page).click(),
  ]);

  expect(download).toBeTruthy();

  // Optional file name check when a name is provided by the browser.
  const suggestedName = download.suggestedFilename();
  expect(suggestedName.length).toBeGreaterThan(0);
});

test('TC_CROP_009 Reset using Clear button', async ({ page }) => {
  await openCropPage(page);
  await uploadFixtureImage(page);

  await expect(clearText(page)).toBeVisible();
  await clearText(page).click();

  // Screenshot point: reset/default state after Clear.
  await expect(originalText(page)).not.toBeVisible();
  expect(await cropValueInputs(page).count()).toBe(0);
  await expect(selectFilesButton(page)).toBeVisible();

  // Secondary assertion only: placeholder text if currently rendered.
  const noImageYet = page.getByText('No image yet', { exact: true });
  if (await noImageYet.count()) {
    await expect(noImageYet.first()).toBeVisible();
  }
});
```

5.2 Test Execution

The automated test cases were executed using the Playwright test runner via the command 'npx playwright test'. The execution was performed in a Chromium browser in headed mode.

All defined test cases were executed successfully without failures, demonstrating that the Crop JPG functionality works as expected under normal conditions.



5.3 Test Report

Playwright automatically generated an HTML test report summarizing the execution results. The report provides detailed information about each test case, including execution time, status, and step-level insights.

All test cases passed successfully, confirming the reliability and correctness of the Crop JPG module.

Playwright Test Report

localhost:9323/#?

Search tests

All 9 | Passed 9 | Failed 0 | Flaky 0 | Skipped 0

Project: chromium

4/19/2026, 3:34:42 PM · Total time: 16.8s

crop_jpg_playwright.spec.js

- TC_CROP_001 Access Crop JPG page chromium — 2.0s
  crop_jpg_playwright.spec.js:93
- TC_CROP_002 Verify initial UI state chromium — 1.2s
  crop_jpg_playwright.spec.js:105
- TC_CROP_003 Open file selector chromium — 1.4s
  crop_jpg_playwright.spec.js:122
- TC_CROP_004 Upload valid JPG image chromium — 1.8s
  crop_jpg_playwright.spec.js:136
- TC_CROP_005 Verify crop value text boxes after upload chromium — 1.5s
  crop_jpg_playwright.spec.js:148
- TC_CROP_006 Edit crop using text boxes chromium — 1.7s
  crop_jpg_playwright.spec.js:166
- TC_CROP_007 Adjust crop region using mouse chromium — 1.9s
  crop_jpg_playwright.spec.js:186
- TC_CROP_008 Download cropped image chromium — 2.4s
  crop_jpg_playwright.spec.js:227
- TC_CROP_009 Reset using Clear button chromium — 1.5s
  crop_jpg_playwright.spec.js:246